

# Programação em Python

# Unidade 3

Desenvolver Aplicações Back-end para Web com Python

# Sessão 02

## Pyhton

### Formatando a saída com *print()*

- Exemplo de saída

*O salário de Maria no mês  
de abril foi R\$ 10500.50*

#### Código Python

```
nome = "Maria"
```

```
mes = "abril"
```

```
salario = 10500.50
```

```
print("O salário de {0} no mês de {1} foi
```

```
R$ { 3 : 2f }").format( nome, mes, salario ) )
```

float (real) :*nf*

*n* – casas decimais com arredondamento

*f* – tipo float

int (inteiro) :*nd*

*n* – espaços ou caractere de preenchimento

*d* – tipo inteiro

Forma Geral:

:*[preenchimento][tipo alinhamento][largura].[precisão]*

*qualquer  
caracter*

*< esquerda  
> direita  
^ centro*

*largura  
mínima  
do campo*

*largura  
máxima  
do campo*

## Python

### Sobre tipos de dados

| Tipos de dados   | Descrição                    |
|------------------|------------------------------|
| Texto            | str                          |
| Numérico         | int, float, complex          |
| Sequência        | list, tuple, range           |
| Mapa             | dict                         |
| Set              | set, frozenset               |
| Lógico           | bool                         |
| Binário (Binary) | bytes, bytearray, memoryview |
| Tipo "None"      | NoneType                     |

## Python

### Strings

- Array de bytes de caracteres Unicode
- Índice baseado em zero – Zero-based
- Comando *for* permite iteração
  - Ex.:

```
for x in "abacaxi"  
    print( x )
```
- Tamanho da string com a função *len()*
  - Ex.:

```
print( len( "abacaxi" ) )
```
- Instrução *in* ou *not in* para verificar a existência de uma string em outra
  - Ex.:

```
print( "ferro" in "ferrovia" )  
print( "casa" not in "ferrovia" )
```



### Saiba mais

- Sobre métodos da String, visite:  
[https://www.w3schools.com/python/python\\_strings\\_methods.asp](https://www.w3schools.com/python/python_strings_methods.asp)
- Sobre UNICODE, visite:  
<https://www.ime.usp.br/~pf/algoritmos/apend/unicode.html>

## Python

### Sobre tipos e conversão de dados (casting)

- Tipo de dado da variável com a função *type()*
  - Ex.:

```
x = 10 // inteiro
print( type ( x ) )
```
  - Resultado: `<class 'int'>`
- Casting de tipos numéricos e strings
  - String
    - Ex.: `print ( "Numero + str( 100 ) )`
  - Integer – *int()*
    - Ex.: `print ( int( "2" ) )`
  - Float – *float()*
    - Ex.: `print ( float( "2.10" ) )`



### Saiba mais

- Sobre tipos de dados, visite:  
[https://www.w3schools.com/python/python\\_datatypes.asp](https://www.w3schools.com/python/python_datatypes.asp)
- Sobre conversão de dados (casting), visite:  
[https://www.w3schools.com/python/python\\_numbers.asp](https://www.w3schools.com/python/python_numbers.asp)

## Pyhton

### Processamento Condicional

- Simples

if condição:

<bloco de código>

<bloco de código pós condição>

- Composta

if condição:

<bloco de código>

else:

<bloco de código>

<bloco de código pós condição>

- Composta aninhada

if condição1:

<bloco de código>

elif condição2:

<bloco de código>

else:

<bloco de código>

<bloco de código pós condição>



## Pyhton

### Processamento Condicional - CASO

- Antes do Python 3.10

`if` condição1:

`<bloco de código>`

`elif` condição2:

`<bloco de código>`

`else:`

`<bloco de código>`

`<bloco de código pós condição>`

- Após o Python 3.10

`match` termo:

`case` pattern1:

`<bloco de código>`

`case` pattern2:

`<bloco de código>`

`case _ :`

`<bloco de código>`

`<bloco de código pós termo>`

## Pyhton

### Processamento Condicional - CASO

- Exemplo com `match`

```
linguagem = input( "Qual linguagem de programação você está aprendida?" )
```

```
match linguagem:
```

```
    case "JavaScript": print( "Você é um desenvolvedor Web" )
```

```
    case "Python": print( "Você é um cientista de dados" )
```

```
    case "PHP": print( "Você é um desenvolvedor back-end" )
```

```
    case "Java": print( "Você é um desenvolvedor massa" )
```

```
    case _: print( "Você é um desenvolvedor" )
```

```
<bloco de código pós condição>
```

## Python

### Tabela Verdade

| Operando1 | Operando2 | E / <b>and</b> | OU / <b>or</b> | Não Operando1 /<br><b>not</b> Operando1 |
|-----------|-----------|----------------|----------------|-----------------------------------------|
| V         | V         | V              | V              | F                                       |
| V         | F         | F              | V              | F                                       |
| F         | V         | F              | V              | V                                       |
| F         | F         | F              | F              | V                                       |

## Pyhton

### Processamento com Repetição (n vezes)

```
for <variável> in <sequência>  
    <bloco de código>  
<bloco de código pós repetição>
```

- <sequência> pode ser uma função `range()` e a variável a ser incrementada ou decrementada a cada passada
- A função `range()` devolve uma lista de valores, incrementados ou decrementados, durante cada ciclo da instrução `for`
- O incremento/decremento padrão do comando `for` é (+ ou - 1)
- O ciclo é baseado em zero (0 – “Zero based”)
- Ex.:
  - `range( 10 )`
  - `range( fim )`
  - `range( [ início ], fim [ , passo ] )`

## Pyhton

### Processamento com Repetição

**while** <condição == True> :

    <bloco de código>

<bloco de código pós repetição>

## Python

### Praticando

#### Problemas:

1. Crie um programa para cada exercício da lista de exercícios de seleção.
2. Crie um programa para cada exercício da lista de exercícios de repetição.

## Python

- Considerações finais.
- Agradecimentos.



# Programação em Python

Botafogo



# Até a próxima!

